

School OS SaaS - Master Product Requirements and Architecture Specification

Document version: 0.2

Status: Pre-database-design requirements baseline

Purpose: Consolidate all product decisions, workflows, features, architecture choices, and design principles agreed before Supabase database design begins.

Change policy: This document is intentionally evolvable. Future decisions should update the version and changelog instead of silently replacing earlier requirements.

Status Legend

Status	Meaning
CONFIRMED	Explicitly agreed as a requirement or architecture decision.
PROPOSED	Recommended during discussion and retained for design review, but not yet fully frozen.
FUTURE	Intentionally postponed while keeping the architecture extensible.
TBD	A detail still needs an implementation decision during the design phase.

Contents

- Executive summary and product vision - Sections 1-2
- SaaS deployment, control plane, and five-engine architecture - Sections 3-5
- School configuration, campuses, academic levels, and academic years - Sections 6-9
- Students, identities, enrollment history, families, authentication, and roles - Sections 10-16
- Approval engine, admissions, curriculum, teachers, attendance, and timetable - Sections 17-23
- Homework, online tests, exams, grading, and results - Sections 24-29
- Fees, discounts, invoices, payments, financial integrity, and overdue handling - Sections 30-36
- Parent/student portals, HR, payroll, leave, library, transport, hostel, and medical - Sections 37-46

- Calendar, documents, storage, notifications, messaging, reports, analytics, search, and audit - Sections 47-56
 - AI, offline capability, Flutter platforms, backup, localization, and module settings - Sections 57-62
 - Onboarding, deployment, migrations, security, data design, and module dependency map - Sections 63-69
 - Deferred and future features - Sections 70-71
 - End-to-end workflows and full decision logs - Sections 72-75
 - Open technical decisions, design guardrails, next phase, and versioning - Sections 76-80
 - Compact product blueprint and product principle - Appendices A-B
-

1. Executive Summary

School OS is a configurable **School Management SaaS platform** for institutions ranging from Play Group/Nursery through Grade 12. The product is not intended to be a simple collection of CRUD forms. The target is a modern school operating system combining academic management, finance, HR, communication, approvals, auditability, automation, offline support, analytics, and AI.

The system will be delivered using a SaaS architecture in which **each customer school has its own Supabase project**, while the projects can live under one Supabase organization controlled by the SaaS operator. This provides strong customer isolation while allowing the same Flutter codebase, database migrations, storage setup, functions, policies, and deployment documentation to be reused for every school.

The school application will support multiple campuses inside a single customer school. Historical academic data must be preserved across years, student promotions, transfers, campus moves, fee changes, attendance corrections, result revisions, and salary changes where appropriate.

The architecture is centered on five engines:

1. **Data Engine** - operational school data and historical records.
2. **Permission Engine** - who can view or perform each action and at what scope.
3. **Workflow Engine** - approvals, corrections, publishing, locking, and exceptional actions.
4. **Automation Engine** - event-driven and scheduled rules, alerts, reminders, and generated actions.
5. **AI Engine** - permission-aware assistants, tutoring, analytics, question generation, and natural-language access to authorized school data.

The next phase after this document is the actual database and backend design.

This document deliberately describes requirements and architecture **without yet defining the final SQL tables**.

2. Product Vision

CONFIRMED

The target product is a **Modern School Operating System**, not only a traditional ERP.

It should combine:

- School and campus administration
- Admissions and enrollment
- Student and family management
- Academic years, classes, sections, rooms, subjects, and curriculum
- Teacher and employee management
- Attendance
- Timetable generation and conflict detection
- Homework, assignments, projects, quizzes, worksheets, and online tests
- Exams, marks, grading, result publication, and report cards
- Fees, discounts, invoices, payments, receipts, overdue handling, and financial controls
- HR, payroll, leave, and employee attendance
- Library
- Transport
- Hostel
- Medical records
- Documents and certificates
- Events and calendars
- In-app, push, and email notifications
- Reporting and analytics
- Permission-aware global search
- Approval and audit workflows
- Offline capability
- AI-based school, teacher, parent, and student features
- Reusable deployment scripts and configuration for commercial SaaS distribution

The system should remain configurable enough that two schools can use very different class structures, fee policies, grading systems, admission formats, curricula, approval hierarchies, and enabled modules without modifying the core database design.

3. High-Level SaaS Deployment Decision

CONFIRMED: Option A - One Supabase Project per School

The selected commercial architecture is:

Supabase Organization (owned by SaaS operator)

```
|
|-- Project: SaaS Control Plane
|
|-- Project: School A
|   |-- Auth
|   |-- PostgreSQL
|   |-- Storage
|   |-- Edge Functions
|   `-- Realtime / policies / migrations
|
|-- Project: School B
|   `-- Independent school backend
|
`-- Project: School C
    `-- Independent school backend
```

Each school project contains only that school's operational data. School A's students, fees, payroll, medical records, and documents do not share tables with School B.

Why this model was selected

- Strong customer isolation
- Simpler Row Level Security than a shared multi-tenant school database
- Easier school-specific backup and restoration
- Easier customer offboarding and migration
- Reduced risk of cross-school data exposure
- Same schema can be reproduced using migration scripts
- Easier troubleshooting per customer
- Allows school-specific capacity or backend upgrades later

Important distinction

There are two different concepts:

- **Supabase Organization:** billing/project container controlled by the SaaS operator.
- **Customer School:** one paying school/customer, normally represented by one Supabase project.

Because one school owns the entire school project, the school database generally does **not** need an `organization_id` column on every operational table. The

project itself is the isolation boundary.

4. SaaS Control Plane

PROPOSED / architecture direction agreed conceptually

A dedicated control-plane project should manage the SaaS business and routing layer, not detailed school records.

Potential control-plane responsibilities:

- Customer/school registry
- School code used before login
- School project reference and routing metadata
- School activation/suspension status
- Subscription/plan metadata if commercial plans are introduced
- Enabled commercial features/modules if licensing requires them
- Application version
- Database schema version
- Last applied migration
- Deployment status
- Central SaaS operator accounts
- Deployment logs
- System-wide service announcements

The control plane should **not** store detailed student, marks, attendance, medical, payroll, or fee records from customer schools.

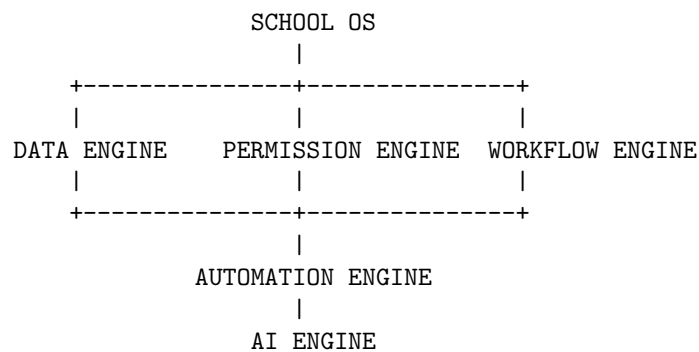
Proposed login routing

```
Open School OS
  |
  v
Enter School Code
  |
  v
Control Plane resolves school project
  |
  v
Connect to that school's Supabase project
  |
  v
Email or Username + Password
  |
  v
Permission-aware school dashboard
```

This allows the same Flutter application to serve many independent school projects.

5. Core Architecture - Five Engines

CONFIRMED



5.1 Data Engine

Responsible for persistent operational data and historical records, including students, employees, academics, finance, attendance, exams, library, transport, hostel, medical, documents, configuration, and related history.

5.2 Permission Engine

Determines both **action permission** and **scope**.

Example actions:

- View
- Create
- Update
- Cancel/reverse
- Approve
- Reject
- Export
- Publish
- Lock/unlock when authorized

Example scopes:

- All campuses
- Assigned campus
- Assigned class

- Assigned section
- Assigned subject
- Assigned students
- Own record
- Own children

A user being able to **see** a record does not automatically mean that user can **modify** it.

5.3 Workflow Engine

Handles approval chains, revision requests, corrections, publishing, locking, transfers, and sensitive operations.

Example:

```

Teacher requests result correction
      |
      v
Old mark + proposed mark + reason
      |
      v
Exam Controller review
      |
      +-- Reject
      |
      |-- Approve
          |
          v
      Revision applied
          |
          v
      Audit entry + notification
  
```

5.4 Automation Engine

Supports immediate, scheduled, and event-triggered actions.

Examples:

- Student absent -> parent notification
- Fee overdue -> reminder and possible late-fee rule
- Result published -> student/parent notification
- Birthday -> optional notification
- Promotion completed -> new enrollment record
- Fee verification approved -> receipt available

5.5 AI Engine

AI features must use the same authorization boundaries as the normal application. AI must never become a path around RLS or application permissions.

6. School and Campus Model

CONFIRMED

Each customer project represents one school organization. A school may contain one or many campuses, for example:

```
School
|
|-- Main Campus
|-- Girls Campus
|-- Junior Campus
`-- User-defined Campus
```

Each campus may have its own:

- Classes
- Sections
- Rooms
- Students
- Teachers
- Fees
- Attendance
- Timetable
- Library activity
- Exams
- Transport assignments
- Other campus-scoped records

Higher-level users may view multiple campuses, while campus-level users operate only within assigned campuses.

A student or teacher can move between campuses, and the system must preserve historical campus assignments rather than overwriting history.

7. School Configuration

CONFIRMED

Each school should be configurable with at least:

- School name
- Short name
- School code
- Logo
- Address
- City
- Country
- Phone
- Email
- Website
- Principal name or linked principal account
- Registration number
- Tax/NTN or equivalent identifier where applicable
- Default academic year
- Currency
- Timezone
- Date format
- Admission-number formatting rules
- Roll-number rules
- Grading rules
- Attendance rules
- Fee rules
- Module settings

School-specific information should be loaded from configuration rather than requiring the developer to hard-code school name and branding throughout the Flutter source.

8. Supported Academic Levels

CONFIRMED

The system must not assume every school has the same class list.

Examples include:

```
Play Group
Nursery
KG-1
KG-2
Grade 1
...
Grade 12
```

Another school may start at Nursery and omit Play Group. Classes/grades are therefore user-configurable.

Each class can have zero, one, or many sections, for example:

```
Grade 8
|-- Section A
|-- Section B
`-- Section C
```

Classes and sections can include:

- Name/code
 - Campus
 - Capacity
 - Room
 - Class teacher
 - Subject-teacher assignments
 - Student roster
 - Timetable
 - Roll-number configuration
-

9. Academic Years and Historical Access

CONFIRMED

Academic years have custom start and end dates. A school may use formats such as:

- 2026
- 2026-27
- Any user-defined display label

Multiple academic years remain available for historical access. One year can be marked as the active/default working year while authorized users can switch to prior years or compare multiple years where the interface supports it.

Historical records must not be deleted when a new academic year begins.

10. Student Core Profile

CONFIRMED

A student profile may include:

- Internal UUID
- School-facing Student ID
- Admission number
- Serial/admission sequence number

- Full name
- Profile photo
- Gender
- Date of birth
- Age as a derived value rather than a permanent manually maintained field where possible
- Nationality
- National ID/B-Form
- Birth certificate
- Blood group
- Father name
- Mother name
- Guardian name when applicable
- Primary contact
- WhatsApp number
- Emergency contact
- Address
- City
- Previous school
- Admission/enrollment date
- Admission class
- Current enrollment summary
- Roll number
- Orphan status
- Disability status
- Student status
- Created timestamp
- Updated timestamp

Student statuses

- Active
- Graduated
- Transferred
- Suspended
- Withdrawn
- Expelled
- Deceased
- Alumni

Status changes must retain history, including date, reason, actor, and notes.

11. Student Identity and Numbering

CONFIRMED

Internal database identifiers and school-facing numbers are separate.

Example:

Internal UUID: 550e8400-e29b-...
Student ID: STU-000124
Admission Number: MPS-2026-00001
Roll Number: 17

Admission-number format is school-configurable. The agreed pattern concept is:

PREFIX - SESSION - SEQUENCE

Examples:

- MPS-2026-00001
- MPS-26-00001
- Custom prefix and separators

The school can choose full-year or two-digit session representation. Sequence length may be configured up to 8 digits.

Roll numbers can be generated sequentially from 1, start from a custom value such as 100 or 1000, or be manually adjusted by authorized users.

12. Enrollment History and Promotion

CONFIRMED

`current_class` and `current_section` must not be the only authoritative historical source.

The system must preserve enrollment history such as:

Academic Year	Campus	Class	Section	Roll No.	Status
2024-25	Junior Campus	Grade 4	A	12	Completed
2025-26	Junior Campus	Grade 5	B	8	Completed
2026-27	Main Campus	Grade 6	A	15	Active

Promotion should be processed through a proper promotion workflow:

Current enrollment
|

```

      v
Results finalized
      |
      +-- Passed -> next class
      +-- Failed -> repeat / school-defined outcome
      `-- Special case -> manual decision
      |
      v
New academic-year enrollment created

```

The old enrollment is retained permanently as history.

Class capacity should be enforced. If a class is full, enrollment should normally be blocked, with an authorized override option if school policy allows it.

13. Family and Parent/Guardian Accounts

CONFIRMED

One family account can be associated with multiple students.

Example:

```

Family Account
|-- Child A - Grade 6
|-- Child B - Grade 3
`-- Child C - Nursery

```

The chosen model is a **shared family login** rather than requiring separate father, mother, and guardian logins. The displayed primary relationship can reflect the real situation: Father, Mother, or Guardian.

Father/mother/guardian access uses the same parent-role capabilities for the linked children, including fees, attendance, results, homework, and other parent-facing information permitted by school policy.

A teacher or employee may also be a parent. One user account can therefore hold multiple application roles and switch between appropriate dashboards/contexts.

Design note: Shared parent credentials reduce account complexity but reduce person-level attribution of which adult physically performed an action. In the agreed first model, the action is attributed to the family/parent account rather than requiring separate adult identities.

14. Authentication

CONFIRMED at product level; implementation details TBD during backend design

Users may log in using:

- Email
- Admin-created username

The system may also store school-generated login IDs and identity metadata.

Because Supabase Auth is natively centered on supported identity providers rather than arbitrary usernames, the exact secure username-to-auth mapping must be finalized in the backend design phase.

Private backend credentials must never be embedded in the Flutter client. The client may use public project connection information as intended by Supabase, while service-role keys, database passwords, email secrets, AI secrets, and management credentials remain server-side.

15. User Roles

CONFIRMED current role catalog

- Super Admin
- School Owner
- Principal
- Vice Principal
- Administrator
- Accountant
- HR
- Teacher
- Coordinator
- Librarian
- Transport Manager
- Driver
- Nurse/Medical Staff
- Student
- Parent/Family Account
- Security Staff
- Maintenance Staff
- Exam Controller

Current decision: A separate IT/Admin school role is not required at this stage.

Roles provide defaults, but authorization should ultimately use permissions and scopes rather than hard-coded role checks alone.

16. Permission and Scope Model

CONFIRMED

Permissions should combine:

ROLE + ACTION + DATA SCOPE + WORKFLOW STATE

Example:

Role: Teacher

Action: Update attendance

Scope: Assigned attendance session/class only

Condition: Session open and not locked

Example action permissions

- View
- Create
- Update
- Request change
- Approve
- Reject
- Cancel/reverse
- Export
- Generate document
- Publish
- Lock/unlock where permitted

Example scopes

- All campuses
- Assigned campuses
- Assigned classes
- Assigned sections
- Assigned subjects
- Assigned students
- Own employee record
- Own family/children

Key role principles

- **Super Admin:** Can be granted broad read/write authority, but actions remain auditable and protected-history deletion is not unilateral.

- **School Owner:** Permission level is school-defined and may include broad read/write capability.
- **Principal:** Broad visibility, analytics, reports, academic oversight, approval authority, and operational capabilities according to policy.
- **Campus Admin:** Manages assigned campus only unless explicitly granted broader scope.
- **Accountant:** Manages fees, payment verification, fee slips/receipts, reports, and other permitted finance work. Cannot silently delete financial records or make protected changes without approval.
- **Teacher:** Sees salary/employee information relevant to self and teaching information. Can perform assigned academic actions but does not receive general fee access.
- **Parent:** Only own linked children/family data.
- **Student:** Only own permitted data.

The school can assign broader permissions to administrative roles such as Owner, Super Admin, Administrator, or Accountant, but teacher/parent/student roles should not be elevated into unrestricted system administrators merely by changing ordinary permissions.

17. Approval and Workflow Engine

CONFIRMED

A central generic approval engine should be reused instead of hard-coding a different approval table for every module.

A request may include:

- Request type
- Requested by
- Target entity/record
- Current/old value
- Proposed/new value
- Reason
- Attachments/evidence if applicable
- Current status
- Approval steps
- Current reviewer
- Decision
- Decision comments
- Created time
- Decision time
- Applied time

Potential uses:

- Fee changes
- Discount changes
- Financial reversals
- Attendance corrections
- Marks corrections
- Student profile corrections
- Salary changes
- Leave approval
- Result publication
- Exceptional history cleanup where legally/operationally permitted

Approval chains are school-configurable. Examples:

Teacher -> Coordinator -> Principal

Teacher -> Principal

Teacher -> Class Teacher only

Accountant -> Principal -> Owner

Rules may also depend on thresholds, for example a larger fee adjustment requiring additional approval.

18. Admission Management

CONFIRMED

The complete potential workflow is:

Application

- > Document Verification
- > Admission Test
- > Interview
- > Approval
- > Admission Fee
- > Enrollment
- > Class/Section Allocation

However, each stage is optional. A school can skip document verification, test, interview, or other steps depending on its policy.

The admission process can generate:

- Application number
- Admission number
- Student ID
- Admission form
- Offer/admission letter
- Student card

- Other school-configured documents

Each stage should retain status and history rather than only storing the current stage.

19. Curriculum and Subjects

CONFIRMED

Schools define their own curriculum. Subjects can vary by class and academic year.

Subject configuration may include:

- Subject name
- Subject code
- Short name
- Type
- Theory component
- Practical component
- Passing requirements
- Credit hours if used
- Applicable classes/sections
- Academic year
- Teacher assignments

Supported subject types can include:

- Compulsory
- Optional
- Elective
- Practical
- Extra-curricular
- Custom

Students may choose different optional subjects within the same grade when the school allows it.

Different sections of the same grade may have different teachers for the same subject.

Example:

```
Grade 8 Mathematics
|-- 8-A -> Teacher A
|-- 8-B -> Teacher B
`-- 8-C -> Teacher C
```

20. Teachers and Employee Profiles

CONFIRMED

Teacher/employee profiles may contain:

- Employee ID
- Name
- Photo
- National/CNIC/ID
- Qualification
- Specialization
- Experience
- Joining date
- Contract type
- Salary information
- Subjects
- Classes
- Sections
- Campuses
- Attendance
- Leave
- Documents
- Department
- Designation
- Performance information

Teachers have their own dashboard with teaching assignments, timetable, attendance responsibilities, assignments, results, notifications, leave, and permitted employee information.

Teachers may teach multiple subjects, classes, sections, and campuses. Access to protected student actions is derived from assigned responsibilities rather than merely from being able to view a campus.

21. Class Teacher and Subject Teacher Model

CONFIRMED

A section may have one designated class teacher while individual subjects are taught by different teachers.

Example:

```
Grade 8-A
|
|-- Class Teacher -> Teacher X
```

```
|-- Mathematics    -> Teacher A
|-- Physics        -> Teacher B
`-- English        -> Teacher C
```

The class teacher designation and subject-teaching assignments are distinct relationships.

22. Attendance

CONFIRMED, with configurable attendance sessions

Attendance must support:

- Manual attendance
- QR attendance using student card
- Present
- Absent
- Leave
- Late arrival
- Early departure
- Attendance percentage
- Parent notification
- Automatic absence alerts
- Monthly reports
- Academic-year reports

The agreed operational model is that official attendance is normally taken in a configured session such as:

- Morning / first-class attendance
- Optional second attendance after break

The school can configure one or both sessions.

A teacher cannot take attendance for another campus or unrelated student group merely because that teacher can see general campus information.

Attendance correction requires approval once a record is considered submitted/locked. The exact same-day edit window can be defined in school settings during design.

QR attendance

Teachers can scan student-card QR codes or mark attendance manually. Basic anti-fraud features such as photo confirmation can be added later; advanced QR anti-fraud is not required in the first version.

22.1 Attendance Capture Source Architecture

CONFIRMED ARCHITECTURE GUARDRAIL / FUTURE-READY

The canonical attendance domain must be independent from the mechanism used to capture attendance. Version 1 supports manual and QR-driven capture, but the database and service boundaries must not assume that these are the only possible sources. Future integrations may include fingerprint/biometric terminals, RFID/NFC readers, camera/face-recognition systems, kiosk devices, and other attendance hardware.

The design principle is:

Capture Source

- > source/device event or adapter
- > identity resolution + validation
- > permission/workflow rules
- > canonical attendance record
- > audit/notification/analytics

This means future devices should not create separate attendance histories or require a destructive rewrite of the attendance tables. The core attendance record should be able to retain or reference source metadata such as capture method, device/event identifier, capture timestamp, external/vendor reference, confidence or validation state where relevant, and sync state where relevant. Exact tables are intentionally deferred to the database design phase.

Manual and QR attendance remain the implementation priority. Biometric, camera/face recognition, RFID/NFC, and advanced anti-fraud mechanisms are **FUTURE** and may require dedicated vendor adapters, device registries, privacy controls, consent/legal review, anti-spoofing, and hardware-specific synchronization.

23. Timetable

CONFIRMED

Timetable management supports:

- Campus
- Class
- Section
- Subject
- Teacher
- Room
- Day/time period

- Breaks
- Teacher availability
- Required subject periods

The system must detect/prevent conflicts such as:

- Teacher assigned to two classes at the same time
- Room assigned to two classes at the same time
- Section/class assigned to overlapping periods

Automatic timetable generation is required, with manual adjustment by authorized users afterward.

24. Homework, Assignments and Learning Work

CONFIRMED

Teachers can create:

- Homework
- Assignments
- Projects
- Worksheets
- Quizzes
- Online tests

Records can include:

- Title
- Instructions
- Class/section/subject
- Due date
- Attachments
- Maximum marks where applicable
- Student submissions
- Teacher feedback
- Late-submission status
- Parent notification

Homework/assignment publication is visible to applicable students and their linked parents.

25. Online Tests and Assessment Activities

CONFIRMED as a supported direction

The platform should support online assessment capabilities such as:

- MCQ
- True/False
- Short answer
- Essay
- Time limit
- Randomized questions where configured
- Automated marking for objective questions
- Attempt history
- Manual marking where needed

Assessment records can coexist with formal examinations, practice tests, quizzes, and assignments.

26. Exams and Examination Types

CONFIRMED

An academic year may contain unlimited/custom assessment events, including:

- Weekly test
- Monthly test
- Practice
- Homework assessment
- Assignment assessment
- Quiz
- First term
- Midterm
- Final term
- Annual exam
- Practical
- Oral
- Custom examination

Reusable exam templates can be created.

A subject may define different component structures, for example:

```
Mathematics: Theory 80 + Practical 20
Computer:    Theory 60 + Practical 40
Another:     Theory 100 only
```

Components are optional and subject-specific.

27. Marks, Result Workflow and Publication

CONFIRMED

Authorized users such as teachers and selected administrative roles may enter/upload marks according to permission.

Result publication follows a controlled workflow:

```
Marks entered
|
v
Submitted / pending review
|
v
Exam Controller reviews
|
|-- Check missing marks
|-- Check errors/anomalies
|-- Review pending modification requests
`-- Review analytics
|
v
Approve for publication
|
v
Publish
|
v
Lock
```

Published/locked results cannot be casually edited. A correction requires a formal change request including old value, new value, reason, reviewer, and approval history.

The system calculates, where configured:

- Total marks
- Percentage
- Grade
- GPA
- Pass/fail
- Subject average
- Class average
- Ranking/position

Ranking can be enabled for:

- Section
- Class

- Grade
- Campus

or disabled completely.

28. Grading Systems

CONFIRMED

Each school defines its own grading rules.

Example:

Range	Grade
90-100	A+
80-89	A
70-79	B
60-69	C
50-59	D
Below 50	F

Another school may use A1/A/B/C/D/E or a completely different model. Grading must therefore be configuration-driven.

29. Result Cards and Academic Documents

CONFIRMED

The system can generate result cards containing configurable information such as:

- School branding
- Student identity
- Academic year
- Class/section
- Subjects
- Marks
- Grades
- GPA/percentage where applicable
- Attendance summary
- Teacher comments
- Signatures/approval areas

The document is generated on demand from stored data. The rendered PDF does not need to be permanently stored unless a future policy explicitly requires it.

30. Fee Types and Fee Structures

CONFIRMED

Supported fee categories include:

- Admission fee
- Tuition
- Examination
- Transport
- Library
- Computer
- Laboratory
- Sports
- Uniform
- Hostel
- Miscellaneous
- Custom categories

Fee structures can depend on:

- Academic year
- Campus
- Class/grade
- Student
- Fee type
- Billing frequency

Billing frequencies include:

- Monthly
 - Quarterly
 - Half-yearly
 - Annual
 - One-time
 - Custom
-

31. Discounts and Scholarships

CONFIRMED

The system supports both reusable policies and student-specific adjustments.

Examples:

- Sibling discount
- Scholarship
- Staff-child discount
- Need-based discount
- Custom discount

Sibling discounts may be enabled or disabled by the school and can use configurable rules.

Discounts can be long-term/recurring or one-time. Sensitive or large adjustments can require approval according to school-defined thresholds.

32. Invoices

CONFIRMED

The system stores the underlying financial obligation and invoice meta-data/history, but **does not store every rendered invoice PDF** in Supabase Storage.

When an authorized user requests an invoice:

Stored fee/invoice data

|

v

PDF/document renderer

|

v

Download / print / share

Invoices can be generated for:

- A specific student
- A selection of students
- All applicable students in a configured batch

Invoice numbers are permanent and follow a pattern such as:

INV-YYYY-MM-NUMBER

The exact sequence rule is configurable during database design.

33. Payments and Parent Payment Evidence

CONFIRMED

Initial payment methods:

- Cash
- Bank transfer
- Wallet transfer

There is no direct online payment gateway in the initial version.

For bank/wallet payments, parents can submit:

- Amount
- Transaction/reference ID
- Bank/wallet information
- Payment date
- Screenshot/evidence

Workflow:

```
Parent submits payment evidence
      |
      v
Pending verification
      |
      v
Accountant review
      |
      +-- Approve -> official payment + receipt
      +-- Reject
      +-- Request correction
```

Duplicate submissions should be flagged using information such as transaction ID, bank/wallet, and amount.

34. Partial, Advance and Overpayments

CONFIRMED

The system supports:

- Partial payments
- Advance payments
- Half-year or annual advance payments
- Allocation of advance payments to future fee periods

Example:

```
Annual payment received
|-- September -> Paid
|-- October   -> Paid
|-- November  -> Paid
`-- ...
```

Overpayments support all four business outcomes:

- Keep as future credit
 - Refund
 - Combination of credit/refund
 - Authorized administrative decision
-

35. Financial Integrity

CONFIRMED

Financial transactions must not be casually deleted.

Instead of destructive deletion:

```
Original financial record
|
v
Correction / reversal / cancellation request
|
v
Reason + approval
|
v
Reversal/correction entry
|
v
Audit trail retained
```

This principle applies to fee changes, payment reversals, and other protected financial history.

36. Fee Overdue Handling

CONFIRMED

Overdue fees can produce:

- Outstanding balance

- Late fee according to configured rules
- Reminder
- Parent notification
- Payment/receipt status updates

Automation rules should make these configurable rather than hard-coded.

37. Parent Portal

CONFIRMED

Parents/family accounts can view permitted data for their linked children, including:

- Attendance
- Results
- Homework
- Assignments
- Timetable
- Fees
- Announcements
- Events
- Teacher messages
- Exam schedule
- School calendar
- Transport
- Library history
- Medical information where policy allows

Parents can:

- Submit payment evidence
- Submit leave requests
- Contact teachers according to communication rules
- Update selected permitted information
- Download generated documents
- Receive notifications

Parents cannot search for or access unrelated students.

38. Student Portal

CONFIRMED

Students have their own accounts and dashboards, including:

- Timetable
- Homework
- Assignments
- Exams
- Results
- Attendance
- Library
- Fees where school policy permits
- Events
- Notices
- Certificates/documents
- Learning materials

Students only access their own data and permitted shared school content.

39. HR and Employee Management

CONFIRMED

The platform includes:

- Employee profile
- Employee attendance
- Leave
- Payroll
- Salary
- Bonuses
- Deductions
- Documents
- Contracts
- Performance
- Department
- Designation

Employee attendance statuses can include:

- Present
 - Absent
 - Late
 - Early departure
 - Leave
 - Half day
 - Overtime
-

40. Payroll

CONFIRMED

Payroll is monthly.

Potential payroll components:

- Basic salary
- Allowances
- Overtime
- Bonus
- Manual deduction
- Deduction reason
- Loan where configured
- Tax where configured
- Net salary

Payroll runs can use a format such as:

MM-YYYY

Finalized payroll records should be locked from casual editing.

Salary changes can be:

- Manually entered
- Configured as an increment after a selected number of months/years
- Auto-increment disabled

Salary changes and sensitive payroll modifications require approval according to policy.

41. Leave Management

CONFIRMED

Student example:

Parent submits leave

|

v

Configured teacher/authority reviews

|

v

Approve / Reject

Employee example:

Employee requests leave

|

v

Configured hierarchy

|

v

Approve / Reject

The school defines the approval chain. Leave types can be configurable, including sick, casual, emergency, annual, maternity, examination, and custom types.

42. Library

CONFIRMED

Library support includes:

- Books
- Book type/category
- Authors/metadata as needed
- Copies/stock
- Student borrowing
- Teacher borrowing
- Purchase of books/items where the school uses this model
- Borrow transaction
- Due date
- Return
- Return status
- Payment status
- Fine where configured
- Lost item status
- Borrowing/purchase history

The first version requires at least basic history for students and teachers.

43. Transport

CONFIRMED for non-GPS functionality; GPS FUTURE

Initial transport features:

- Vehicle information
- Driver

- Route
- Stops
- Pickup point
- Drop-off point
- Transport fee
- Student assignment
- Bus attendance
- Parent notification

FUTURE: Live GPS tracking and advanced location services.

44. Hostel

CONFIRMED, optional module

Hostel management supports:

Hostel

- > Building
 - > Floor
 - > Room
 - > Bed

and:

- Student allocation
- Hostel fee
- Check-in/out
- Hostel attendance
- Visitor records

Schools without a hostel can disable the module.

45. Medical / Health

CONFIRMED, sensitive module

Potential data:

- Blood group
- Allergies
- Emergency contact
- Medical notes
- Vaccination records
- Medication alerts

- Nurse visits
- Medical incidents

Medical data requires stricter permissions than ordinary student profile data. Nurse/medical staff, parents, and selected senior roles may receive appropriate access; unrelated roles such as accountants should not receive medical access.

46. Inventory

FUTURE

Full inventory/asset management is intentionally postponed. The architecture should not prevent it from being added later.

Potential future scope includes computers, desks, projectors, laboratory equipment, sports equipment, vehicles, suppliers, maintenance, assignment, and disposal.

47. Events and Calendar

CONFIRMED

The school calendar can contain:

- Exams
- Holidays
- Parent-teacher meetings
- Sports day
- Annual functions
- Trips
- Workshops
- Birthdays
- Staff meetings
- Admission deadlines
- Custom events

Events can trigger notifications according to school configuration.

48. Certificates and Generated Documents

CONFIRMED

On-demand documents include:

- Student ID card
- Staff ID card
- Admission letter
- Bonafide certificate
- Character certificate
- Transfer certificate
- Leaving certificate
- Result card
- Fee receipt
- Salary slip
- Enrollment certificate
- Other templates added later

Generated document files should normally be created from database data and downloaded/printed/shared immediately rather than stored permanently in Storage.

49. Uploaded Documents and Supabase Storage

CONFIRMED

Important uploaded source documents are stored, for example:

- Student photo
- Birth certificate
- National/student identity documents
- Teacher contract
- Employee documents
- Other important evidence

Uploads should be reduced/compressed to a maximum of approximately **1 MB per file** under the current requirement.

Images should be resized/compressed before storage. PDF handling should also enforce the size limit; implementation may reject or compress oversized PDFs depending on the chosen client/server processing strategy.

Storage should use structured paths, for example:

```
students/{student_id}/photo/...
students/{student_id}/documents/...
employees/{employee_id}/contracts/...
school/branding/...
```

Storage access must be protected by appropriate policies.

50. Notifications and Communication

CONFIRMED

Initial channels:

- In-app notifications
- Push notifications
- Email

FUTURE: WhatsApp integration.

Notifications support:

- Immediate events
- Scheduled reminders
- Event-triggered notifications

Each user should have a notification center with:

- Read/unread state
- Mark as read/all read
- History
- Linked action or record where appropriate

Initial email delivery may use configurable SMTP/app-password credentials. The architecture should allow later replacement with Google Workspace, transactional email services, or other providers without redesigning school data.

TBD: The exact push-notification provider is not frozen. FCM was not selected as a firm requirement in the discussion.

51. Internal Messaging

CONFIRMED as desired product capability

Internal communication can support school-to-parent, teacher-to-parent, and administrative communication subject to permissions.

Examples:

- School announcement to all parents
- Teacher message to parents of assigned students
- Administrative message to staff

Messaging rules should prevent a teacher from browsing or contacting unrelated students/families without authorization.

52. Reports

CONFIRMED

Reports should support:

- PDF
- Excel
- CSV
- Print

The system should support predefined reports and eventually flexible/custom reports, for example:

Students in Grade 8 with attendance below 75% and outstanding fees.

Report content must respect the requesting user's permissions.

53. Analytics and Dashboards

CONFIRMED

Role-specific dashboards include:

Principal / senior administration

- Student counts
- Attendance trends
- Fees and collections
- Outstanding balances
- Academic results
- Teacher/staff indicators
- Enrollment trends
- Dropout/withdrawal trends
- Campus comparisons
- Alerts and pending approvals

Teacher

- Today's classes
- Attendance responsibilities
- Students
- Assignments
- Upcoming exams
- Marks/result tasks

Accountant

- Collections
- Outstanding balances
- Pending payment verification
- Financial reports

Parent

- Child performance
- Attendance
- Fees
- Homework
- Notifications

Student

- Personal academic dashboard

Dashboard customization is desired so users can choose/reorder permitted widgets later.

54. Global Search

CONFIRMED

The application should offer permission-aware global search.

An administrator searching a student's name might be able to find the profile, attendance, results, fees, documents, and related records according to permission.

A teacher receives only authorized academic/student information. A parent cannot search other students. Search must apply the same authorization constraints as direct navigation.

55. Audit Logging

CONFIRMED

Sensitive actions and important operational changes must be logged. Audit information can include:

- Actor
- Action

- Entity/record
- Timestamp
- Old value
- New value
- Reason
- Request/approval reference
- Device/session metadata where appropriate
- IP/network metadata where appropriate and lawful

Examples:

- Fee changed
- Result corrected
- Attendance corrected
- Salary modified
- User permission changed
- Payment reversed
- Protected record archived

Super Admin actions are also audited.

Audit logs cannot be freely erased by the actor who created them.

56. Data Deletion, Archiving and Retention

CONFIRMED policy direction

Different data classes require different treatment.

Financial data

No normal destructive deletion. Use reversal/cancellation/correction with history.

Attendance

Historical records are retained. Corrections preserve revision/audit history.

Published results

Historical results are retained. Corrections use an approved revision process.

Audit logs

No normal UI deletion.

General operational data

Soft-delete/archive where appropriate.

Exceptional cleanup

Eligible non-protected history may have a controlled purge process, but not as a unilateral Super Admin action. It requires reason, approval, and auditability. Core protected financial/audit/academic history remains subject to stricter retention rules.

AI conversations

Authorized administration may delete stored AI conversation history to manage storage, subject to future retention policy.

57. AI Features

CONFIRMED

Planned AI capabilities include:

- AI Teacher Assistant
- AI Student Tutor
- AI Homework Assistant
- AI Question Generator
- AI Result Analysis
- AI Early Warning
- AI Parent Assistant
- AI Principal/School Analytics
- Natural-language queries over authorized school data

Examples:

- Teacher: “Create a Grade 7 mathematics quiz about fractions.”
- Student: “Explain photosynthesis in simpler language.”
- Principal: “Which Grade 8 students have declining performance?”
- Parent: “How many days was my child absent this month?”

AI conversations can be stored for continuity. Admins may clear them according to retention/storage policy.

Hard security rule

AI can only access data the requesting user is already permitted to access. It must not receive a service-role path that bypasses user authorization for convenience.

58. Offline Capability

CONFIRMED at architecture level

Flutter clients must support selected offline workflows. Attendance is a primary use case.

Example:

```
Device downloads authorized working data
      |
      v
Internet unavailable
      |
      v
Teacher records attendance locally
      |
      v
Local encrypted storage
      |
      v
Connection restored
      |
      v
Synchronization + conflict detection
```

Important records should not silently use a naive last-write-wins strategy. Conflicts should be detected and, where necessary, resolved by an authorized user.

Local cached sensitive data should be encrypted.

TBD during technical design: Exact offline scope for homework, marks, timetable, and broader student information, and the precise sync/conflict algorithm for each data type.

59. Flutter Client Platforms

CONFIRMED

Initial application targets:

- Mobile application using Flutter, with Android as the current development target
- Windows desktop application distributed as **.exe**

A Flutter web application is not the current selected PC target.

60. Backup and Disaster Recovery

CONFIRMED

The product should plan for:

- Database backups
- Configuration backups
- Complete deployment/recovery documentation
- School data export where appropriate
- Migration/version tracking

Backup responsibilities should distinguish Supabase-managed infrastructure backup capabilities from application-level exports and configuration snapshots.

61. Localization and Currency

CONFIRMED

Initial UI language: **English**.

The architecture should permit later localization such as Urdu, Arabic, or other languages without redesigning core data.

Supported currencies should include:

- PKR
- SAR
- USD
- Other configured currencies

Each school normally selects its default currency.

62. Module Enable/Disable Capability

PROPOSED and strongly aligned with the SaaS direction

Modules should be capable of being enabled or disabled per school, for example:

School A

- Attendance: Enabled
- Fees: Enabled

- Exams: Enabled
- Library: Enabled
- Hostel: Disabled
- Transport: Disabled
- AI: Enabled

This supports schools that do not use certain modules and creates a future path for subscription tiers without requiring separate codebases.

63. School Setup and Onboarding

PROPOSED

A new-school setup wizard can eventually configure:

1. School identity and branding
2. Currency/timezone/date settings
3. First Super Admin
4. Campuses
5. Academic year
6. Classes and sections
7. Subjects/curriculum
8. Grading system
9. Admission-number rules
10. Fee categories/rules
11. Roles/approvals
12. Enabled modules

This is preferable to editing source code separately for every customer.

64. Deployment Repository and Reproducibility

CONFIRMED objective

All database and backend creation steps must be stored in source control so a new school project can be created reproducibly.

Recommended repository structure:

```
school_os/  
|  
|-- flutter_app/  
|  
|-- supabase/  
|   |-- migrations/
```

```

|    |-- functions/
|    |-- seed/
|    |-- storage/
|    |-- policies/
|    `-- config/
|
|-- control_plane/
|
`-- documentation/

```

Database tables, functions, policies, storage rules, seed/default configuration, and setup steps should be migrations/code rather than undocumented manual changes in a Supabase dashboard.

65. Version and Migration Management

PROPOSED and recommended for SaaS operations

After many schools are deployed, projects may temporarily run different schema versions. The system should therefore track:

- App version
- Database schema version
- Last migration
- Migration status
- Deployment date

Example:

```

School A -> schema v1.8
School B -> schema v1.8
School C -> schema v1.7

```

A controlled migration process is required before commercial scale.

66. Security Principles

CONFIRMED direction

1. Use Supabase RLS as a primary database-level authorization layer.
2. Do not trust client-side UI hiding as security.
3. Public client configuration is not treated as a secret; privileged keys remain server-side.
4. Sensitive medical, financial, payroll, and employee documents receive stricter policies.

5. Every privileged workflow is scoped by campus/class/user context.
6. Sensitive changes use approval or revision processes.
7. Historical integrity is preferred over destructive updates/deletes.
8. Offline cached sensitive data is encrypted.
9. Global search and AI follow normal authorization.
10. Super Admin does not mean unaudited.

67. Data Design Principles for the Next Phase

CONFIRMED principles, not final SQL design

The database should be relational and normalized enough to preserve history and enforce permissions.

The system should **not** create one giant **students** table containing attendance, fees, class history, medical information, and results.

Conceptually:

```
Student
|
|-- Identity/Profile
|-- Family relationships
|-- Enrollment history
|-- Attendance
|-- Subjects
|-- Assessments/results
|-- Fees/payments
|-- Documents
|-- Medical
|-- Transport
|-- Hostel
`-- Library
```

Likewise, employee salary history, teacher assignments, permissions, approval requests, notifications, and other histories should be modeled as appropriate related records rather than overwritten fields.

68. Initial School Project Module Map

CONFIRMED scope direction

```
FOUNDATION
|
```

```
|-- Authentication / Accounts
|-- People / Profiles
|-- Roles / Permissions / Scopes
|-- School Configuration
|-- Campuses
|-- Academic Years
|
ACADEMICS
|
|-- Classes
|-- Sections
|-- Rooms
|-- Subjects
|-- Curriculum
|-- Teacher Assignments
|-- Timetable
|-- Attendance
|-- Homework / Assignments
|-- Exams
|-- Results
|
STUDENT LIFE
|
|-- Admissions
|-- Students
|-- Families
|-- Enrollment History
|-- Leave
|-- Library
|-- Transport
|-- Hostel
|-- Medical
|
FINANCE & HR
|
|-- Fee Structures
|-- Discounts
|-- Invoices
|-- Payments
|-- Receipts
|-- Employee Management
|-- Employee Attendance
|-- Payroll
|-- Salary History / changes
|
PLATFORM ENGINES
```

```
|  
|-- Approval / Workflow  
|-- Audit  
|-- Notifications  
|-- Automation  
|-- Documents  
|-- Search  
|-- Analytics / Reports  
|-- AI  
`-- Offline Sync
```

69. Dependency Order for Database Design

The proposed implementation order is:

1. Foundation and common database conventions
2. Authentication/account mapping and people profiles
3. Roles, permissions, scopes, and authorization strategy
4. School configuration and campuses
5. Academic years
6. Classes, sections, and rooms
7. Subjects and curriculum
8. Students, families, and enrollment history
9. Employees and teachers
10. Teacher/class/subject assignments
11. Timetable
12. Attendance
13. Homework and assignments
14. Exams, marks, grading, and results
15. Fees, discounts, invoices, and payments
16. HR, payroll, and leave
17. Library
18. Transport
19. Hostel
20. Medical
21. Documents/storage policies
22. Approval/workflow engine integration
23. Audit engine
24. Notifications and automation
25. Reports/search/analytics
26. AI integration
27. Offline synchronization
28. Backup/deployment/versioning validation

The exact ordering can be adjusted as dependencies become clearer during schema design.

70. Current Non-Goals / Deferred Features

The following are not first-priority requirements, although the architecture should not block them:

- Live bus GPS tracking
 - Advanced QR anti-fraud/identity confirmation
 - Full inventory/asset-management module
 - WhatsApp integration
 - Direct online payment gateway
 - RFID/NFC/fingerprint/biometric attendance
 - Camera / face-recognition attendance devices and recognition services
 - IoT/smart classroom integration
 - CCTV integration
 - Blockchain certificate verification
 - Government reporting integrations
 - Advanced external accounting integrations
 - Additional languages beyond English
-

71. Potential Future Advanced Features

FUTURE

- GPS/live bus tracking
- RFID/NFC attendance
- Fingerprint / biometric attendance devices
- Camera / face-recognition attendance systems
- Smart classrooms / IoT
- Digital learning library
- More advanced e-learning
- Expanded online examinations
- Voice assistant
- Advanced AI tutoring
- AI-generated teaching plans
- Predictive analytics
- Advanced early-warning models
- External APIs
- Government reporting
- Accounting integrations

- WhatsApp messaging
 - Payment gateways
 - Multiple languages/RTL
 - Inventory and asset lifecycle
 - Digital certificate verification
-

72. Key End-to-End Workflows

72.1 New Student Admission

Application

- > Optional verification/test/interview stages
- > Approval
- > Admission fee if required
- > Student identity created
- > Family linked
- > Enrollment created
- > Class/section allocation
- > Student ID/card generated
- > Parent/student access activated

72.2 Annual Promotion

Academic year closes

- > Results verified/finalized
- > Promotion decision
- > Next-year enrollment generated
- > New class/section/roll number
- > Prior enrollment remains historical

72.3 Attendance

Teacher opens authorized attendance session

- > Manual or QR marking
- > Submit
- > Absence automation
- > Parent notification
- > Record locks according to policy
- > Correction requires approval when protected

72.4 Fee Payment by Bank/Wallet

Fee obligation exists

- > Parent uploads transfer evidence

- > Pending verification
- > Accountant review
- > Approve / reject / correction requested
- > Official payment created if approved
- > Permanent receipt number
- > Receipt rendered on demand

72.5 Result Publication

Marks entered

- > Teacher/admin submission
- > Validation and missing-mark checks
- > Exam Controller review
- > Pending corrections resolved
- > Approval
- > Publish
- > Lock
- > Parent/student notification

72.6 Sensitive Change

User requests protected modification

- > Old/new values captured
- > Reason required
- > Approval chain resolved
- > Approver reviews
- > Change applied only if approved
- > Audit entry written
- > Notification generated if configured

73. Requirements Decision Log - Initial Questionnaire

This section records the major questionnaire outcomes in compact form.

A. Institution and academic structure

- School product supports Play Group/Nursery through Grade 12, but levels are configurable.
- A school may omit Play Group or use a different class structure.
- Multiple sections per class are supported.
- Academic years preserve historical records.
- One school can have multiple campuses.

- Campus data and actions are scope-aware.

B. Roles

Agreed role families include senior administration, finance, HR, teaching, support services, students, and family accounts. Permissions are granular and scope-aware rather than simple role-only checks.

C. Students and families

- Rich student profile supported.
- One family account can link multiple children.
- Employee/teacher can also be a parent using one multi-role account.
- Student statuses include Active, Graduated, Transferred, Suspended, Withdrawn, Expelled, Deceased, and Alumni.

D. Admissions

- Full multi-stage admission supported.
- Stages are optional.
- Application/admission numbers, IDs, forms, letters, and cards can be generated.

E. Classes and promotion

- Classes, sections, rooms, capacities, timetables, and student lists supported.
- Promotion creates new enrollment history rather than overwriting the old class.

F. Curriculum

- School-defined subjects and curriculum.
- Compulsory/optional/elective models supported.
- Student-level optional subject selection supported.

G. Teachers

- Complete teacher profile and dashboard.
- Multiple subjects/classes/sections/campuses supported.

H. Attendance

- Manual and QR.
- Configurable morning and optional post-break sessions.
- Reports and parent notifications.
- Protected corrections require approval.

I. Timetable

- Conflict detection required.
- Automatic generation plus manual adjustment.

J. Homework and assignments

- Homework, assignments, projects, worksheets, quizzes, online tests.
- Due dates, attachments, marks, submissions, feedback, late status, notifications.

K. Exams/results

- Unlimited/custom exam types.
- Flexible assessment components.
- Automatic calculations.
- Configurable grades and ranking.
- Exam Controller approval before publication.
- Published results lock and correction workflow.

L. Fees

- Multiple fee types and billing frequencies.
- Discounts/scholarships.
- Partial/advance/overpayments.
- Bank/wallet evidence verification.
- No destructive financial deletion.
- On-demand invoices/receipts; rendered PDFs not stored by default.

M/N. Parent and student portals

- Both have dedicated accounts/dashboards.
- Access is limited to own/linked records.

O/P. Communication

- In-app, push, email initially.
- WhatsApp later.
- Internal communication should respect relationships and permissions.

Q. Library

- Borrowing and purchase-oriented history for students/teachers.

R. Transport

- Vehicles, drivers, routes, stops, assignments, attendance, fees, notifications.
- GPS later.

S. Hostel

- Optional full allocation model.

T. Medical

- Sensitive student health information with restricted access.

U. Inventory

- Deferred.

V/W. HR and payroll

- Full employee management.
- Monthly payroll.
- Overtime, bonus, manual deductions/reasons.
- Salary changes configurable and approval-controlled.

X. Leave

- Student and employee leave with school-defined approval chains.

Y. Calendar

- School events, exams, holidays, meetings, trips, deadlines, etc.

Z. Certificates/documents

- Generated on demand, normally not stored as rendered files.

AA. AI

- Teacher, student, parent, principal, question generation, analysis, early warning.
- Permission-aware data access.

AB/AC. Analytics and reports

- Role dashboards, analytics, export, custom reports.

AD. Security

- Approval, audit, historical integrity, strict access control.

AE/AF. Language and currency

- English initially, localization-ready.
- PKR/SAR/USD/other currencies.

AG. Payments

- No direct gateway initially; cash plus verified bank/wallet evidence.

AH. Platforms

- Flutter mobile + Windows `.exe`.

AI. Offline

- Offline capability required with secure local storage and conflict-aware synchronization.

AJ/AK. Distribution and branding

- Product is now SaaS.
- One Supabase project per customer school.
- School settings/branding should be configuration-driven.

AL. Automation

- Automation engine is part of the core architecture.

AM. Future features

- Advanced hardware, GPS, integrations, languages, and other enhancements remain future-ready.

AN. Delivery philosophy

- Build a broad platform architecture, but implement in dependent phases rather than as one giant undifferentiated schema.

74. Detailed Decision Log - Q50 to Q100

Question	Agreed decision
Q50	Authentication supports email and admin-created username; additional school login identifiers may be stored.
Q51	One login can have multiple roles, e.g. Teacher + Parent.
Q52	Father/mother/guardian use a shared family account for linked children.
Q53	Display primary family relationship based on circumstance; first model does not require separate adult login attribution.
Q54	Separate action permissions and data scopes; campus/class/subject/own-record scoping.
Q55	Attendance is a configured official session (morning and optionally after break), not arbitrary attendance across every viewed campus.
Q56	Different sections of the same subject/grade may have different teachers.
Q57	A section can have one class teacher while each subject can have a different teacher.
Q58	Academic-year start/end dates are school-defined.
Q59	Historical years remain accessible; active-year working mode supported.
Q60	Important profile changes should preserve meaningful history/audit.
Q61	Student status changes preserve date, reason, actor, and notes.
Q62	Admission stages have status/history and can be skipped by school configuration.
Q63	Admission number is configurable using school prefix, session, and up-to-8-digit sequence.
Q64	Roll numbering supports automatic/custom starting numbers/manual adjustment.
Q65	Class capacity is enforced with authorized override if policy allows.

Question	Agreed decision
Q66	Promotion uses a formal process and creates new enrollment history.
Q67	Subjects have configurable metadata/components/passing rules.
Q68	Curriculum can differ completely by class.
Q69	Individual students can choose optional subjects.
Q70	Unlimited/custom assessment events are supported.
Q71	Reusable exam templates are supported.
Q72	Marks can be entered by authorized roles; Exam Controller approval is required before publication.
Q73	Rendered invoice PDFs are not stored; underlying financial data/history is stored.
Q74	Invoice numbers are permanent, e.g. INV-YYYY-MM-number.
Q75	Verified payments have permanent receipt numbers.
Q76	Payment evidence workflow supports approve/reject/request correction.
Q77	Duplicate payment evidence should be detected/flagged.
Q78	Payroll is monthly and finalization/locking is required.
Q79	Salary increment behavior is configurable/manual and can be disabled; salary changes should be historically/auditably represented during design.
Q80	Leave approval hierarchy is school-configurable.
Q81	Uploaded files should be compressed/limited to about 1 MB.
Q82	Storage uses structured paths by entity/person/document type.
Q83	Notifications support immediate, scheduled, and event-triggered delivery.

Question	Agreed decision
Q84	Initial email may use app-password/SMTP configuration; provider can be replaced later.
Q85	Exact push provider remains TBD; do not freeze FCM as the requirement.
Q86	Important offline conflicts should be detected rather than silently overwritten.
Q87	Offline sensitive data should be encrypted.
Q88	Plan for database, configuration, and complete deployment/recovery backups.
Q89	SaaS direction adopted; later finalized as one Supabase project per school.
Q90	AI cannot bypass user permissions.
Q91	AI conversation history is stored; authorized admin may delete it for retention/storage.
Q92	Global search respects permissions; parents cannot search unrelated students.
Q93	Audit logs are protected; users cannot erase their own history.
Q94	Financial/attendance/result/audit history follows non-destructive or revision-based policies.
Q95	Owner and Super Admin permissions are school-defined and can be broad; administrative roles may receive elevated permissions, while teacher/parent/student remain constrained.
Q96	Separate IT/Admin role not required currently.
Q97	Principal has broad operational visibility plus approval and analysis responsibilities.
Q98	Super Admin is audited; protected historical deletion is not unilateral and exceptional cleanup requires approval/reason.
Q99	English first, localization-ready architecture.

Question	Agreed decision
Q100	Product direction is Modern School Operating System using the five-engine architecture.

75. Confirmed SaaS Decision After Q100

After the requirements questionnaire, the commercial deployment choice was finalized as:

Option A: one Supabase project per customer school.

Current SaaS-level structure:

```
One Supabase Organization
|
|-- SaaS Control Plane Project
|-- Customer School Project A
|-- Customer School Project B
|-- Customer School Project C
`-- ...
```

The same source code, migrations, policies, storage configuration, functions, seed data, and documentation should be reusable to deploy a new school.

76. Open Technical Decisions for the Design Phase

The following do **not** block this requirements baseline but must be finalized during implementation design:

1. Exact Supabase Auth strategy for username login.
2. Exact push-notification provider.
3. Exact offline scope by module and conflict-resolution rules per record type.
4. Exact SQL schema, keys, indexes, constraints, enums, and naming conventions.
5. Exact RLS policy implementation and whether some permissions are materialized/cached.
6. Exact approval-engine storage model and dynamic approver resolution.
7. Exact document-rendering technology in Flutter/backend.
8. Exact email sending architecture and secret handling.
9. Exact AI provider/service layer and safe retrieval architecture.

10. Exact school-project provisioning automation and whether Supabase Management API automation is used at launch.
 11. Exact subscription/pricing plans and module limits, if commercial tiers are introduced.
 12. Exact retention durations for logs, uploaded documents, and AI conversations.
 13. Exact backup/export tooling exposed to school administrators versus SaaS operators.
 14. Exact format and verification strategy for QR student cards.
 15. Exact rules for same-day attendance edits before approval becomes mandatory.
-

77. Requirements That Must Not Be Lost During Database Design

These are architectural guardrails:

- Preserve academic-year and enrollment history.
 - Preserve student campus moves.
 - Do not implement permissions as role-name checks only.
 - Separate view scope from modification authority.
 - Do not allow teachers to access unrelated fee records.
 - Do not allow parents to search unrelated students.
 - Do not let AI bypass authorization.
 - Do not destructively delete financial history.
 - Do not silently overwrite published results.
 - Do not silently overwrite protected attendance corrections.
 - Audit Super Admin activity.
 - Generate routine PDFs on demand instead of filling Storage with duplicate rendered documents.
 - Store important original uploaded documents securely and with size limits.
 - Keep one customer school isolated in its own Supabase project.
 - Keep all schema and infrastructure changes reproducible through migrations/configuration.
 - Design for offline conflicts instead of assuming perfect connectivity.
 - Keep configuration flexible for different school structures and policies.
-

78. Next Phase - Database and Backend Design

The next phase should begin with **Foundation + Authentication + People + Roles/Permissions**, then progress through dependent modules.

The design phase should produce, at minimum:

- Naming conventions
- Schema/module boundaries
- Table list and responsibilities
- Primary and foreign keys
- Junction/relationship tables
- Historical tables/revision strategy
- Constraints and validation rules
- Indexing strategy
- RLS policy matrix
- Role/permission/scope model
- Approval-engine model
- Audit model
- Storage buckets and policies
- Database functions/triggers where justified
- Realtime usage rules
- Edge Functions where server-only operations are required
- Offline sync identifiers/versioning strategy
- Seed/default configuration
- Migration order
- Deployment scripts
- Backup and rollback approach
- Test plan for authorization and financial integrity

No production SQL should be considered complete until it is tested against the business workflows captured in this document.

79. Documentation Maintenance and Versioning

This file is the initial master requirements baseline.

Recommended future versions:

v0.1 - Consolidated requirements before database design
v0.2 - Database architecture added
v0.3 - RLS/permission architecture finalized
v0.4 - Workflow/audit/automation architecture finalized
v0.5 - MVP implementation specification
v1.0 - First production baseline

Each update should include:

- Version
- Date
- Sections changed

- Reason for change
- Decision owner/reviewer if needed

This keeps product decisions, code, and database behavior aligned as the platform grows.

80. Changelog

v0.2

- Added explicit future attendance capture architecture: biometric/fingerprint terminals, camera/face-recognition devices, RFID/NFC readers, and additional hardware must integrate through capture adapters/events rather than changing the canonical attendance record.
- Added device/source metadata as a design-time extensibility requirement while keeping these integrations deferred.
- Confirmed the Codex/VS Code documentation handoff and project-context hierarchy.

v0.1

- Consolidated product vision from the full requirements discussion.
 - Recorded the school/campus academic model.
 - Recorded roles, permissions, scopes, workflows, approvals, and audit principles.
 - Recorded student, academic, attendance, examination, finance, HR, library, transport, hostel, medical, communication, reporting, AI, and offline requirements.
 - Recorded SaaS deployment choice: one Supabase project per customer school.
 - Recorded control-plane concept and reproducible migration/deployment requirement.
 - Identified deferred features and unresolved technical decisions.
-

Appendix A - Compact Product Blueprint

```
SCHOOL OS SaaS
|
|-- SaaS Control Plane
|   |-- School registry / routing
|   |-- Deployment/version state
|   `-- Future subscription/module licensing
```

```

|
|-- One Supabase Project per School
|
|   |-- Foundation
|   |   |-- Accounts / people
|   |   |-- Roles / permissions / scopes
|   |   |-- School config / campuses
|   |   `-- Academic years
|   |
|   |-- Academics
|   |   |-- Classes / sections / rooms
|   |   |-- Subjects / curriculum
|   |   |-- Teacher assignments
|   |   |-- Timetable
|   |   |-- Attendance
|   |   |-- Homework / assignments
|   |   `-- Exams / results
|   |
|   |-- Students
|   |   |-- Admissions
|   |   |-- Families
|   |   |-- Enrollment history
|   |   |-- Library
|   |   |-- Transport
|   |   |-- Hostel
|   |   `-- Medical
|   |
|   |-- Finance / HR
|   |   |-- Fees / discounts
|   |   |-- Invoices / payments / receipts
|   |   |-- Employees
|   |   |-- Attendance / leave
|   |   `-- Payroll
|   |
|   |-- Platform Engines
|   |   |-- Workflow / approvals
|   |   |-- Audit
|   |   |-- Automation
|   |   |-- Notifications
|   |   |-- Search / reports / analytics
|   |   |-- Documents / storage
|   |   |-- AI
|   |   `-- Offline sync

```

Appendix B - Product Principle

Every major action should be evaluated as more than a database update.

For example, changing a published mark should conceptually be:

Authenticated user

- > Permission check
- > Scope check
- > Workflow-state check
- > Change request
- > Reason + old/new value
- > Required approval
- > Authorized update/revision
- > Audit log
- > Notification/automation

That same design philosophy should be reused for finance, attendance, results, salary, student status, leave, and other protected operations.

End of School OS SaaS Master Product Requirements and Architecture Specification v0.1